

Session 19 : Physique, GPU et IA

Objectifs de la session

- Comprendre la **séparation des rôles** : la physique simule, la politique d'IA décide.
- Maîtriser le vocabulaire du **Reinforcement Learning** : état, action, récompense, retour, politique, valeur.
- Distinguer le RL de la **neuroevolution** — deux familles d'algorithmes, deux façons d'améliorer une politique.
- Concevoir des **observations** et une **fonction de récompense** qui décrivent correctement une tâche de mouvement.
- Comprendre ce qu'est un **modèle du monde** : apprendre la physique au lieu de la calculer — et pourquoi l'erreur s'accumule.
- Comprendre quand un **compute shader** accélère une simulation, et quand il ne l'accélère pas.
- Savoir où chercher pour aller plus loin : articles fondateurs, moteurs et outils.

🔗 Le fil conducteur

Les sessions précédentes ont construit le **monde physique** : forces, contacts, joints, véhicules, requêtes de scène. Cette session change **qui calcule quoi**.

Trois idées, dans l'ordre :

1. **La physique ne décide pas de l'objectif**. Un moteur physique sait calculer la conséquence d'une force ; il ne sait pas vouloir marcher.
2. **Une politique d'IA choisit les commandes**. Elle peut être écrite à la main, apprise par renforcement, ou obtenue par évolution.
3. **Le GPU change le lieu du calcul**. Pour des milliers d'éléments indépendants, il peut tout calculer sans repasser par le CPU.

Dans les trois cas, la démarche du cours ne change pas : définir un état, une règle de mise à jour, une mesure du résultat — puis vérifier.

Partie 1 : Physique et IA — qui fait quoi ?

Une politique pilote, la physique répond

Une animation pré-enregistrée rejoue une trajectoire décidée à l'avance. Une politique d'IA **observe** la situation courante et **choisit** une commande. Le moteur physique applique cette commande et calcule le mouvement qui en résulte.

Composant	Responsabilité
Capteurs	Mesurer l'état utile : vitesse, orientation, contacts, distances aux obstacles.
Politique π	Transformer les observations en commandes : accélérer, tourner, freiner, actionner un joint.
Moteur physique	Appliquer forces et contraintes, intégrer le mouvement, résoudre les contacts.
Évaluateur	Mesurer si le comportement atteint l'objectif : progression, stabilité, collisions, énergie.

Exemple. Un personnage ne reçoit pas une position à afficher : il choisit des couples articulaires ; le ragdoll actif (Session 18), la gravité et les contacts produisent la démarche.

L'IA n'est pas la physique

Si l'agent demande une accélération irréalisable, le moteur ne téléporte pas le personnage : les forces, l'inertie, les contacts et les limites d'articulation restent en vigueur. Inversement, un moteur physique ne sait pas **quel objectif poursuivre** sans contrôleur.

Cette séparation sert au débogage :

- L'agent prend une mauvaise décision → inspecter observations, politique, récompense.
- Le mouvement ne respecte pas la mécanique → inspecter le modèle physique, les collisions, le pas de temps.

Trois façons d'obtenir une politique

Approche	Principe	Exemple typique
Écrite à la main	Règles explicites : « tourner vers le côté le plus dégagé »	Voiture arcade, NPC simple
Apprise par RL	Ajuster la politique à partir des récompenses reçues	Marche, manipulation, conduite
Obtenue par évolution	Faire évoluer une population de politiques	Petits réseaux, contrôleurs de créature

Les trois coexistent en production. Une règle écrite à la main est souvent le meilleur **point de comparaison** pour vérifier qu'un apprentissage apporte vraiment quelque chose.

Deux jalons historiques — le contrôle procédural. **Simbicon** (Yin, Loken, van de Panne, 2007) fait marcher un bipède avec des automates à états finis et un contrôleur de pas : aucune IA statistique, mais une politique **conçue**. C'est le point de départ intellectuel de la locomotion physique.

DeepMimic (Peng et al., 2018) apprend la même chose par RL, en imitant des captures de mouvement. L'objectif n'est plus d'écrire le contrôleur, mais de décrire une **récompense d'imitation**.

Entre les deux : dix ans, et un déplacement du travail — de la conception du contrôleur vers la conception de la récompense.

Partie 2 : Reinforcement Learning

Le formalisme, en cinq symboles

Le RL décrit le problème comme une boucle :

1. L'agent reçoit une **observation** o_t de l'état du monde.
2. Sa **politique** π choisit une **action** a_t .
3. Le monde évolue : nouvel état, et une **récompense** r_t .
4. L'algorithme ajuste π pour augmenter les récompenses futures.

$$o_t \rightarrow [\pi]a_t \rightarrow [\text{physique}](o_{t+1}, r_t)$$

Symbole	Signification
s_t	État complet du monde (tout ce qui détermine la suite).
o_t	Observation : ce que l'agent perçoit — souvent un sous-ensemble de s_t .
a_t	Action : commande envoyée au monde (force, couple, braquage, cible de joint).
r_t	Récompense scalaire : note immédiate de l'action.
$\pi(a \mid o)$	Politique : distribution de probabilité sur les actions, ou fonction déterministe.

Observation $\neq$ état. Le personnage ne « voit » pas les vitesses internes de ses articulations : il en reçoit une mesure choisie par le programmeur. Cette distinction est la source de nombreuses erreurs de conception.

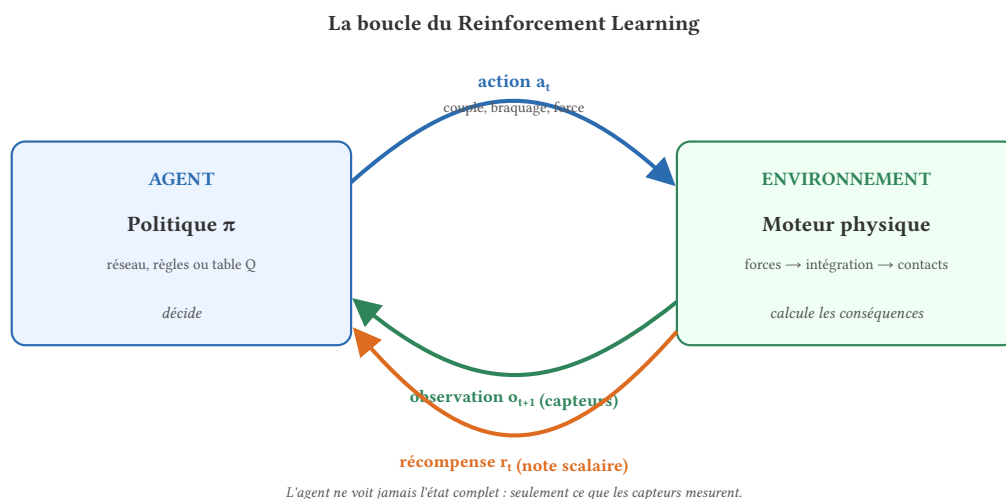


Figure 1: La boucle du RL. L'agent choisit une action ; le moteur physique calcule les conséquences ; les capteurs renvoient une observation et l'évaluateur une récompense. Les trois flèches ne portent pas la même information : l'action est **choisie**, l'observation est **mesurée**, la récompense est **attribuée par le programmeur**.

Le retour : ne pas être myope

Maximiser la récompense **immédiate** ne suffit pas. Un agent qui vise seulement la note du pas courant peut apprendre à rester immobile dans une zone rentable. On optimise donc le **retour**, somme pondérée des récompenses futures :

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$$

$\gamma \in [0, 1]$ est le **facteur d'actualisation** : il règle l'importance du futur.

γ	Comportement induit
0	Agent purement myope : seule la récompense immédiate compte.

γ	Comportement induit
0.9	Horizon d'environ une dizaine de pas — typique pour des tâches courtes.
0.99	Horizon long : l'agent accepte un détour pour un objectif lointain.
1	Aucune préférence pour le présent ; risque de retour infini sur les épisodes cycliques.

Exemple numérique. Avec $\gamma = 0.9$, une récompense de +1 obtenue dans 10 pas vaut $0.9^{10} \approx 0.35$ aujourd'hui : la moitié environ de sa valeur future. Avec $\gamma = 0.99$, elle vaut $0.99^{10} \approx 0.90$. C'est ce qui décide si l'agent « attend » ou « fonce ».

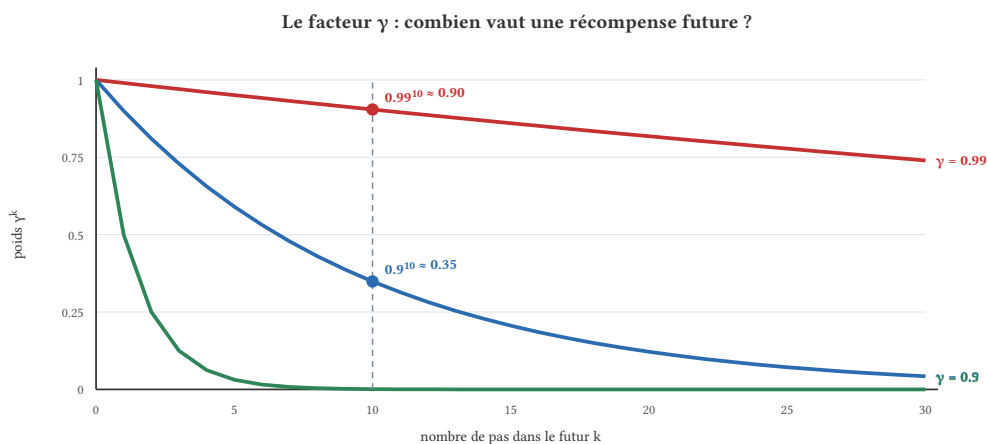


Figure 2: Poids γ^k d'une récompense située k pas dans le futur. À $\gamma = 0.5$ (vert), l'agent ne voit pas au-delà de quelques pas. À $\gamma = 0.9$ (bleu), l'horizon utile est d'une dizaine de pas. À $\gamma = 0.99$ (rouge), une récompense à 10 pas garde 90 % de son poids — l'agent accepte un long détour.

Valeur d'un état, valeur d'une action

On définit la **valeur** d'un état sous une politique π : l'espérance de retour en partant de là.

$$V^\pi(s) = \mathbb{E}[G_t \mid s_t = s, \pi]$$

Et la **valeur d'une action** dans un état — c'est-à-dire « si je fais a maintenant, puis je suis π » :

$$Q^\pi(s, a) = \mathbb{E}[G_t \mid s_t = s, a_t = a, \pi]$$

Ces deux fonctions sont liées par l'équation de **Bellman**, qui exprime une idée simple : la valeur d'un état = récompense immédiate + valeur actualisée de la suite.

$$Q^\pi(s, a) = \mathbb{E}[r_t + \gamma \max_{a'} Q^\pi(s', a')]$$

Si l'on connaît Q , la politique optimale est immédiate : **choisir l'action de plus grande valeur**. Tout l'apprentissage consiste à estimer Q sans connaître le modèle du monde.

Q-learning à la main — un couloir de 4 cases. Prenons un couloir : S0 - S1 - S2 - S3, avec S3 l'arrivée. Deux actions : L et R. Récompense +1 en entrant dans S3, sinon -0.02 par pas. On apprend avec $\alpha = 0.5$, $\gamma = 0.9$, et une table Q initialisée à 0.

La règle de mise à jour (Q-learning) est :

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Première mise à jour — depuis S2, action R, on atteint S3 : $r = +1$, et S3 est terminal donc $\max Q(s', a') = 0$.

$$Q(S2, R) \leftarrow 0 + 0.5[1 + 0.9 \times 0 - 0] = 0.5$$

Deuxième mise à jour — depuis S1, action R, on atteint S2 : $r = -0.02$, et $\max_a Q(S2, a) = 0.5$ (l'action R qu'on vient d'apprendre).

$$Q(S1, R) \leftarrow 0 + 0.5[-0.02 + 0.9 \times 0.5 - 0] = 0.5 \times 0.43 = 0.215$$

Transition	r	$\gamma \max Q(s')$	Q après mise à jour
S2 → S3 (R)	+1.00	0.00	0.500
S1 → S2 (R)	-0.02	0.45	0.215
S0 → S1 (R)	-0.02	0.194	0.087

Après propagation, la politique gloutonne est « toujours R » : c'est le chemin le plus court vers S3. En quelques lignes de table, on vient de faire du RL.

Ce que montre l'exemple : la valeur « remonte » le couloir depuis l'arrivée. C'est exactement ce que fait un algorithme d'apprentissage sur un problème plus grand — mais avec un réseau de neurones au lieu d'une table, parce que l'espace d'états d'un personnage est continu et immense.

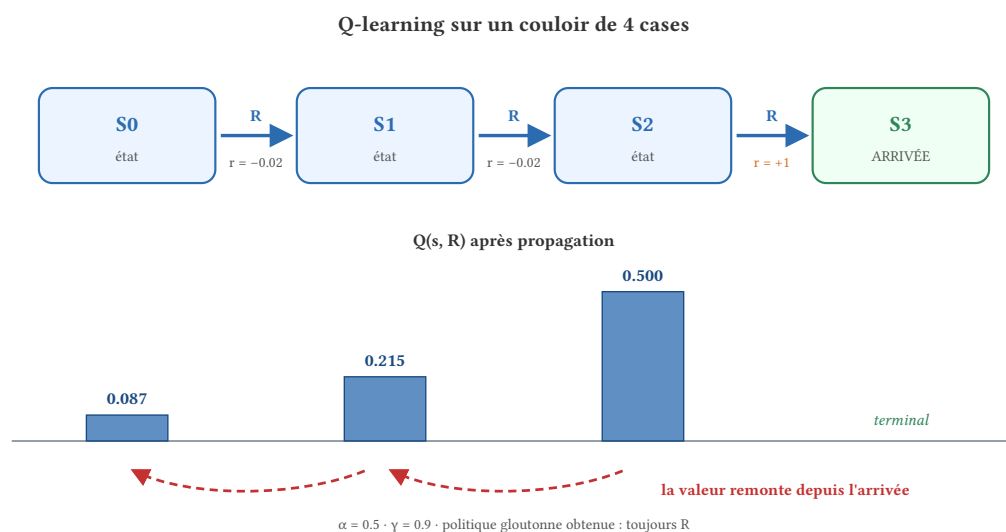


Figure 3: Les trois mises à jour de l'exemple, et la valeur $Q(s, R)$ obtenue. La barre de S0 est la plus courte : la récompense de l'arrivée est encore loin, donc fortement actualisée. Les flèches rouges rappellent le sens de propagation — **de l'arrivée vers le départ**, pas l'inverse.

Explorer ou exploiter

Un agent doit choisir entre **utiliser** ce qu'il sait déjà (exploitation) et **essayer** autre chose (exploration). Une politique purement gloutonne se bloque dans la première solution trouvée, même médiocre.

Stratégie	Principe
ϵ -glouton	Avec probabilité ϵ , tirer une action au hasard ; sinon, prendre la meilleure. ϵ décroît avec le temps.
Bruit gaussien	Ajouter un bruit continu à l'action — adapté aux commandes réelles (couple, braquage).
Entropie	Ajouter un terme qui récompense les politiques variées, pour éviter de se figer trop tôt.

Exemple. $\epsilon = 0.3$ en début d'apprentissage, puis décroissance jusqu'à 0.01 : le début du couloir est exploré largement, la fin est exploitée finement.

Familles d'algorithmes RL (culture générale)

Famille	Idée	Quand on la croise
Méthodes tabulaires	Estimer $Q(s, a)$ dans une table	Problèmes discrets, pédagogie
Deep Q-Network	Un réseau estime Q , entraîné sur des transitions rejouées	Jeux à actions discrètes
Policy gradient	Optimiser directement π en montant le gradient du retour	Actions continues
PPO / TRPO	Policy gradient avec contrainte de taille du pas	Locomotion, référence pratique
Actor-critic	Une politique et une estimation de valeur	Standard moderne
Model-based	Apprendre un modèle du monde puis planifier	Robotique, contrôle optimal

Le choix dépend surtout de la **nature des actions** : discrètes ou continues. Une commande de couple articulaire est continue — on croise donc surtout du policy gradient en animation physique.

Concevoir la récompense : un exemple détaillé

Récompense naïve « avancer le plus loin possible » : l'agent apprend à se jeter au sol et à glisser. Il faut exprimer le comportement visé par **plusieurs termes**.

Terme	Ce qu'il encourage	Signe
Progression	Vitesse vers l'avant projetée sur la direction cible	+
Stabilité	Torse vertical, cap maintenu	+
Hauteur	Rester à hauteur de marche, ni rampant ni sautillant	+

Terme	Ce qu'il encourage	Signe
Effort	Limiter l'amplitude des couples articulaires	—
Secousses	Limiter les à-coups (variation des couples)	—
Chute	Pénaliser la fin d'épisode sur une chute	—

On combine : $r = w_p r_{\text{prog}} + w_s r_{\text{stab}} - w_e r_{\text{effort}} - w_j r_{\text{secousse}} - r_{\text{chute}}$.

Les coefficients w ne sont pas des détails : doubler w_s peut transformer un coureur en statue. C'est un **choix de conception**, et il se règle en regardant des épisodes, pas en raisonnant dans l'abstrait.

Comparer deux politiques par leur fitness. Prenons une fitness volontairement simple, où la progression est une fraction de circuit entre 0 et 1 :

$$F = 100d_{\text{frac}} - 1.0t_s - 30N_{\text{coll}} - 50 \text{ sortie}$$

Deux agents évalués sur le même épisode :

Agent	d_{frac}	t_s	N_{coll}	Sortie	F
A — rapide, brutal	1.00	42	3	non	$100 - 42 - 90 = -32$
B — prudent	0.75	30	0	non	$75 - 30 = +45$

B gagne malgré une progression moindre. C'est le point central de la conception de récompense : les pénalités **façonnent** le comportement, et un agent rapide mais destructeur peut être moins bon qu'un agent lent et propre.

Si l'on veut un agent rapide **et** propre, il faut réduire le poids du temps tout en gardant les pénalités — ou imposer la propreté comme **contrainte** (épisode terminé si collision), plutôt que comme simple pénalité.

⚠ Reward hacking — l'agent optimise ce qu'on mesure

L'agent n'apprend pas ce qu'on **voulait** ; il apprend ce qu'on a **écrit**. Quelques formes classiques :

- **Progression euclidienne** vers un point : l'agent traverse le mur, ou s'éloigne puis revient.
- **Vitesse sans stabilité** : l'agent se projette en avant et tombe en boucle.
- **Rester en vie sans objectif** : si survivre rapporte, l'agent apprend à ne rien faire.
- **Exploiter un bug de collision** : une zone qui donne une récompense en boucle devient un refuge.

La parade n'est pas algorithmique, elle est **expérimentale** : visualiser plusieurs épisodes, isoler chaque terme de la récompense, et tester sur des situations qui n'ont pas servi à l'apprentissage.

Conditions d'un entraînement interprétable

- **Épisodes réinitialisables** : mêmes conditions initiales pour comparer deux politiques ; plusieurs graines aléatoires pour mesurer la robustesse.

- **Pas de temps fixe** : une politique appelée à une fréquence variable rend les résultats difficiles à reproduire.
- **Séparation entraînement / évaluation** : la piste d'entraînement sert à apprendre, une piste différente sert à juger.
- **Budget explicite** : nombre d'épisodes, de pas ou de secondes de simulation — sinon on ne sait pas ce qu'a coûté le résultat.

💡 Pourquoi le RL est coûteux en animation

Une politique de locomotion nécessite des millions de pas de simulation. Une seule simulation interactive, à 60 Hz, est beaucoup trop lente : une heure de calcul ne couvre qu'une heure de mouvement.

D'où trois accélérateurs, tous vus ailleurs dans le cours :

1. **Parallélisme** : simuler des milliers de personnages en même temps (sur CPU multithread ou GPU).
2. **Simplification** : entraîner sur un modèle réduit, valider sur le modèle complet.
3. **Imitation** : partir d'une capture de mouvement pour guider l'exploration (DeepMimic).

Partie 3 : Neuroevolution

Le réseau comme génome

En neuroevolution, on traite les paramètres d'un réseau comme un **génom** : un vecteur de nombres réels. On évalue une **population** de génomes en simulation, puis on crée une nouvelle génération à partir des meilleurs.

1. **Initialiser** N génomes par des tirages aléatoires (petits poids).
2. **Évaluer** chaque génome sur un ou plusieurs épisodes identiques.
3. **Sélectionner** les meilleurs — éventuellement en conservant des **élites** intactes.
4. **Muter** : $w_{i'} = w_i + \varepsilon$, avec ε un petit bruit aléatoire.
5. Facultativement **croiser** deux parents en combinant leurs paramètres.
6. Répéter jusqu'à un budget de générations ou de pas de simulation.

Boucle complète, en pseudo-code.

```
population <- initialiser N génomes
répéter pour chaque génération :
  pour chaque génome :
    réinitialiser l'environnement (même départ)
    exécuter la politique pendant un épisode
    fitness <- mesurer le résultat
  trier la population par fitness
  elites <- les E meilleurs, conservés tels quels
  parents <- sélection(parents par tournoi)
  population <- elites + mutation(parents)
retourner le meilleur génome
```

Détail qui compte : l'environnement doit être réinitialisé à l'**identique** pour tous les génomes d'une génération. Sinon on ne compare pas des politiques, on compare des chances.

Opérateurs de sélection

Opérateur	Principe	Effet
Troncature	Garder les k meilleurs	Simple, converge vite, diversité faible
Tournoi	Tirer k génomes, garder le meilleur	Pression réglable par k
Roulette	Probabilité proportionnelle à la fitness	Favorise les extrêmes
Élitisme	Copier les meilleurs sans mutation	Garantit de ne pas régresser

L'**élitisme** est le garde-fou le plus utile : sans lui, une bonne solution peut disparaître par malchance à la génération suivante.

Mutation, en chiffres. Un génome de trois poids : $w = (0.42, -0.18, 0.77)$. Probabilité de mutation par poids $p = 0.10$, bruit $\varepsilon \sim \mathcal{N}(0, 0.1)$.

Poids	Valeur	Tirage	Mutée ?	Nouvelle
w_1	0.42	0.04	oui	0.30
w_2	-0.18	0.61	non	-0.18
w_3	0.77	0.08	oui	0.81

Détail des deux mutations : $w_1 : 0.42 + 0.1 \times (-1.2) = 0.30$ et $w_3 : 0.77 + 0.1 \times 0.4 = 0.81$. Un tirage **inférieur** à $p = 0.10$ déclenche la mutation ; au-dessus, le poids est recopié tel quel.

Le **taux** de mutation contrôle le pas d'exploration : à $p = 0.5$, la moitié des poids bougent à chaque génération et la population n'accumule rien ; à $p = 0.01$, l'évolution est très lente.

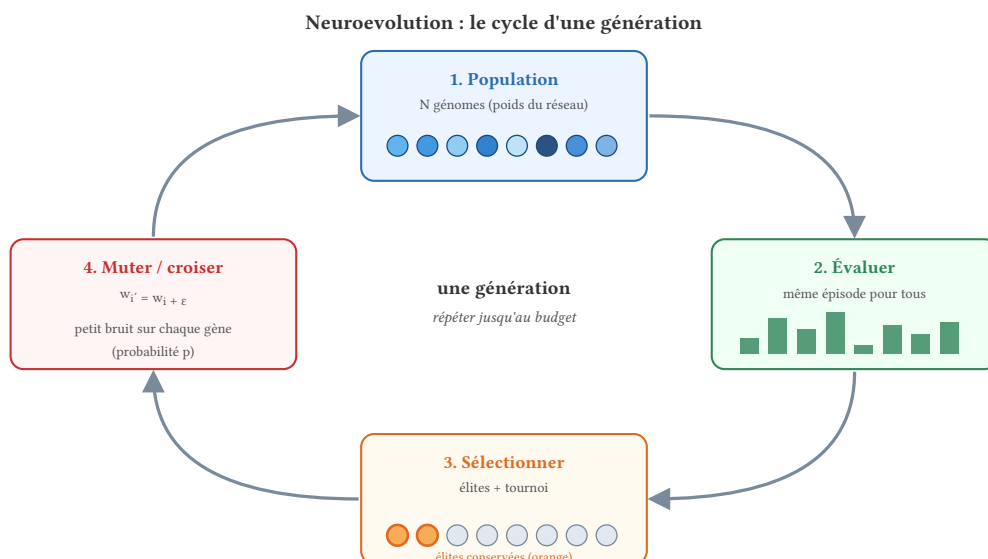


Figure 4: Une génération complète. Les points du haut sont les génomes ; les barres vertes, leur fitness sur le même épisode ; les points du bas, la population de la génération suivante — les deux orange sont les élites, recopiées sans mutation. Le cycle se répète jusqu'au budget de calcul.

Croisement — et pourquoi ce n'est pas magique

Le croisement combine deux parents. Deux variantes simples :

- **Uniforme** : chaque gène vient du parent 1 ou 2, à pile ou face.
- **Un point** : on coupe le vecteur à une position k et on échange les deux morceaux.

Attention : croiser deux réseaux de neurones ne combine pas deux « idées » — les poids n'ont pas de sens indépendant. Le croisement aide quand les gènes sont relativement indépendants ; il peut aussi détruire une bonne combinaison. En pratique, beaucoup d'implémentations n'utilisent **que** mutation et élitisme, et fonctionnent.

NEAT — faire évoluer aussi la topologie

Dans les approches classiques, la **structure** du réseau est fixée par le programmeur et seuls les poids évoluent. NEAT (Stanley & Miikkulainen, 2002) fait évoluer **aussi** la topologie : ajout de neurones, ajout de connexions, avec un mécanisme de **spéciation** qui protège les innovations récentes de la concurrence immédiate.

C'est un jalon important : il montre que la structure du réseau est elle-même un objet de conception, pas une évidence. Pour un cours d'introduction, on garde une topologie fixe — mais il faut savoir que ce n'est pas la seule option.

Diversité et convergence prématurée

Une population qui converge vers un seul génome a perdu sa capacité d'explorer. Le symptôme est net : la fitness plafonne, et toutes les mutations produisent des agents quasi identiques.

Trois remèdes simples :

- **Augmenter le taux de mutation** quand la fitness stagne : cela redémarre l'exploration.
- **Injecter quelques génomes aléatoires** à chaque génération (« immigrants ») pour entretenir un fond de diversité.
- **Évaluer sur plusieurs épisodes**, à des conditions différentes : deux génomes qui semblent équivalents sur un seul départ divergent souvent sur cinq.

C'est l'analogue évolutionnaire du dilemme exploration/exploitation du RL — mais à l'échelle de la population entière, pas d'un seul agent.

RL et neuroevolution : ne pas les confondre

Question	RL	Neuroevolution
Que compare-t-on ?	Une politique, mise à jour à partir des retours d'expérience	Une population de politiques, classées par fitness
Comment apprend-on ?	Gradient estimé sur des trajectoires	Sélection, mutation, éventuellement croisement
Information utilisée	Récompense à chaque pas, souvent exploitée finement	Souvent une seule note par épisode

Question	RL	Neuroevolution
Atout	Échantillon-efficace quand ça marche	Simple, robuste, parallélisable, aucune dérivée
Limite	Réglage délicat, instabilité, récompense à concevoir	Très coûteux en évaluations
Vocabulaire	Politique, valeur, retour, γ	Génome, population, fitness, génération

La neuroevolution est une **famille de méthodes** ; elle n'est pas synonyme du RL moderne. Elle reste très utilisée en robotique évolutionnaire et pour des contrôleurs de petite taille, où sa simplicité et sa robustesse compensent le coût.

Le même problème, vu par les deux familles. **Problème** : un bras à deux articulations doit atteindre une cible.

- **Version RL** : l'agent reçoit à chaque pas la distance à la cible et une récompense $r = -d$. Il apprend une politique continue par gradient.
- **Version neuroevolution** : on évalue 100 réseaux sur la même cible ; la fitness est $-d_{\text{finale}}$. On garde les 10 meilleurs, on mute.

La version évolutionnaire n'a besoin d'aucune notion de retour ni de γ — elle ne voit que le résultat final. C'est plus simple à expliquer, mais elle jette l'information des pas intermédiaires. Pour une cible atteinte en 30 pas, c'est un gaspillage acceptable ; pour un problème à horizon long, c'est un handicap sérieux.

Partie 4 : Capteurs, observations et réseau

Une IA ne reçoit pas « le monde »

Le programme choisit et **encode** les observations. Ce choix détermine ce que l'agent peut apprendre. Pour un personnage ou un véhicule, une observation typique contient :

- des **distances** mesurées par raycast (géométrie locale),
- des **vitesse**s (linéaire et angulaire),
- une **orientation relative** à un objectif,
- parfois des **contacts** (telle roue touche-t-elle le sol ?).

Observation	Ce qu'elle apporte	Piège
Raycasts	Distance au premier obstacle dans une direction	Ne dit rien sur ce qui est derrière
Vitesse	Permet d'anticiper le freinage ou l'inertie	Amplitude très variable selon l'unité
Erreur d'orientation	Indique la direction à corriger	Discontinuité à $\pm 180^\circ$
Contacts	Sait si une roue porte ou si le pied touche	Binaire : peu d'information graduelle

Règle pratique : normaliser chaque entrée dans une plage comparable — par exemple $[0, 1]$ pour les distances, $[-1, 1]$ pour les angles et les vitesses réduites. Sinon une entrée de grande amplitude domine numériquement les autres.

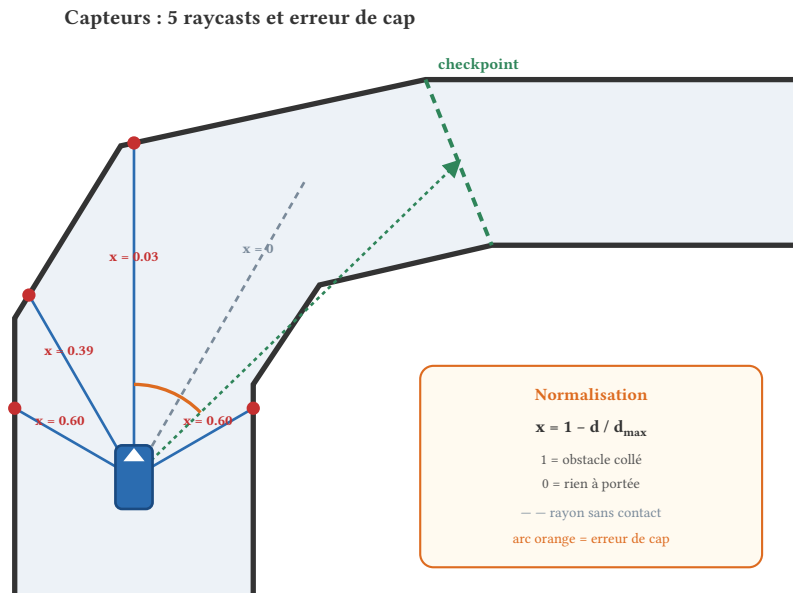


Figure 5: Les cinq raycasts et l'erreur de cap, dans un virage. Chaque rayon part du véhicule dans une direction fixe **relative au véhicule** et renvoie la distance au premier obstacle : les valeurs x sont normalisées entre 0 (rien à portée) et 1 (obstacle collé). L'arc orange est l'angle à corriger — il indique au réseau **de quel côté** tourner, ce que les seules distances ne disent pas.

Normaliser un raycast. Un rayon mesure une distance d bornée par sa longueur maximale $d_{\max}$. On envoie au réseau :

$$x = 1 - d / d_{\max}$$

Ainsi $x = 1$ signifie « obstacle collé », $x = 0$ signifie « rien à portée ». Cette convention a un avantage pratique : une entrée proche de 1 signale un danger **proche**, ce qui est cohérent avec l'intuition de beaucoup de fonctions d'activation.

Vérification : si $d_{\max} = 8\text{m}$ et $d = 6\text{m}$, alors $x = 1 - 6/8 = 0.25$ — le passage est largement dégagé.

Réseau dense minimal : $7 \rightarrow 8 \rightarrow 2$

Une couche cachée suffit pour visualiser le principe. Chaque neurone caché calcule une somme pondérée puis une activation :

$$h_j = \tanh\left(\sum_i w_{ji}x_i + b_j\right)$$

Puis la sortie :

$$y_k = \tanh\left(\sum_j v_{kj}h_j + c_k\right)$$

Le nombre de paramètres se calcule directement :

$$(7 \times 8 + 8) + (8 \times 2 + 2) = 64 + 18 = 82$$

82 nombres réels. C'est le génome complet. Une population de 32 agents, c'est donc 2624 nombres à évaluer — ce qui donne une idée très concrète de ce qu'on appelle « faire évoluer un réseau ».

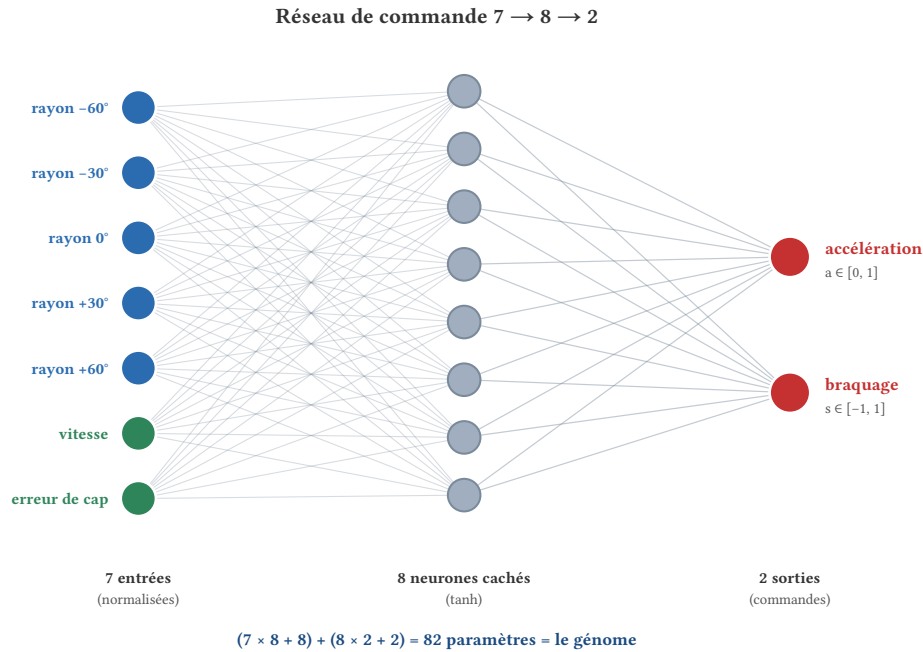


Figure 6: Le réseau de commande. Les cinq entrées bleues sont les raycasts, les deux vertes l'état du véhicule. Chaque connexion porte un poids : 82 au total, c'est-à-dire exactement ce que la mutation et le croisement modifient. Les sorties ne pilotent pas le mesh — elles deviennent des commandes physiques bornées.

Passe avant, en chiffres. Réduisons à 2 entrées → 2 neurones cachés → 1 sortie pour que l'arithmétique tienne à l'écran.

Entrées : $x = (0.8, -0.5)$. Poids de la couche cachée : $w_1 = (0.5, -0.3)$, $b_1 = 0.1$; $w_2 = (0.2, 0.7)$, $b_2 = -0.2$.

$$h_1 = \tanh(0.5 \times 0.8 + (-0.3) \times (-0.5) + 0.1) = \tanh(0.65) \approx 0.572$$

$$h_2 = \tanh(0.2 \times 0.8 + 0.7 \times (-0.5) - 0.2) = \tanh(-0.39) \approx -0.371$$

Sortie, avec $v = (0.4, 0.9)$ et $c = 0.05$:

$$y = \tanh(0.4 \times 0.572 + 0.9 \times (-0.371) + 0.05) = \tanh(-0.055) \approx -0.055$$

Étape	Somme pondérée	Après activation
Neurone caché 1	0.65	$h_1 = 0.572$
Neurone caché 2	-0.39	$h_2 = -0.371$
Sortie	-0.055	$y = -0.055$

Deux remarques. D'abord, tanh sature : une somme de 5 donne 0.9999 — au-delà d'environ 3, augmenter l'entrée ne change presque plus la sortie. Ensuite, la sortie est bornée : il faut la **remapper** vers la plage physique de l'action (couple, braquage) plutôt que de l'envoyer telle quelle.

Choisir une activation

Fonction	Plage	Usage typique
$\tanh$	-1 à $+1$	Couche cachée et sorties continues signées (braquage)
Sigmoïde	0 à 1	Sorties bornées positives (accélérateur)
ReLU	0 à $+\infty$	Réseaux profonds ; peu utile pour un petit contrôleur

Pour un contrôleur physique, des sorties bornées sont précieuses : elles évitent d'avoir à découper des commandes aberrantes, ce qui introduirait des non-linéarités que l'agent n'a pas apprises.

Du réseau à la commande

La sortie du réseau n'est pas appliquée au mesh : elle est **traduite** en commandes physiques, puis le moteur fait le reste.

Sortie réseau	Traduction physique
$a \in [0, 1]$	Fraction de la force moteur maximale, appliquée au sol (cf. Session 17)
$s \in [-1, 1]$	Braquage, multiplié par l'angle maximal de roue
Couples articulaires	Envoyés aux moteurs de joint (cf. Session 18, active ragdoll)

Le réseau ne contourne jamais la physique. Il propose ; le moteur dispose.

Partie 5 : Physique + apprentissage — cas d'usage

Locomotion

Le cas d'école. Le personnage doit avancer sans tomber, sur un terrain parfois irrégulier. Le contrôleur envoie des couples articulaires ; la récompense combine progression et stabilité.

Difficulté principale : l'exploration. Un réseau aléatoire tombe immédiatement, donc ne reçoit presque aucune récompense de progression. D'où l'intérêt de l'**imitation** (partir d'une capture de mouvement) ou d'un **curriculum** (apprendre d'abord à se tenir debout).

Ragdoll actif — le lien avec la Session 18

La Session 18 a montré un ragdoll **actif** : une simulation physique où des moteurs de joint ramènent le personnage vers une pose cible. Ici, la pose cible n'est plus écrite à la main : elle est **produite** par une politique.

Version	Source des commandes
Active ragdoll (S18)	Pose cible choisie par le code, moteurs PD vers cette pose

Version	Source des commandes
Ragdoll piloté par IA	La politique choisit les cibles ou les couples, la physique exécute

C'est la même machinerie physique. Seul le cerveau change — et c'est précisément le message de la session.

Véhicules et conduite

Un véhicule est un excellent banc d'essai : le modèle physique est simple (Session 17), les observations sont naturelles (raycasts, vitesse, orientation), et le comportement est facile à juger visuellement.

On peut y comparer trois politiques sur la **même** piste : écrite à la main, apprise par RL, obtenue par évolution. C'est le meilleur moyen de comprendre ce que chaque approche apporte.

Corps mous et tissus pilotés

Un tissu ou une chevelure n'a pas besoin d'être **piloté** : il est déjà simulé par contraintes (Sessions 8 et 18). L'IA intervient quand il faut **agir** sur un objet déformable — saisir un tissu, gonfler un ballon, contracter un muscle.

Dans ce cas, la politique ne commande plus des articulations rigides, mais des paramètres de la simulation : longueurs de repos, pression, raideur. C'est un domaine plus difficile, où le gradient est moins direct.

Sim-to-real et randomisation de domaine

Une politique entraînée en simulation se comporte mal dans le monde réel si la simulation est trop propre. La **randomisation de domaine** consiste à faire varier pendant l'entraînement ce qu'on ne peut pas modéliser exactement :

- masse et inertie du personnage,
- frottement du sol,
- délai d'action (le temps entre la décision et son effet),
- bruit des capteurs.

L'agent apprend alors une politique **robuste** à ces variations plutôt qu'une politique taillée pour une simulation idéale. C'est une technique de **robustesse**, pas d'optimisation — et elle coûte du temps d'entraînement.

Pourquoi le parallélisme est central

Le RL consomme énormément de simulation. Trois leviers, du plus simple au plus lourd :

Levier	Principe
Accélération du temps	Exécuter plusieurs pas physiques par frame affichée
Mondes multiples sur CPU	Simuler N personnages en parallèle, un par cœur

Levier	Principe
Mondes multiples sur GPU	Simuler des milliers de copies, les données restant sur la carte

C'est ici que la frontière entre « IA » et « GPU » se brouille : le goulot d'étranglement de l'apprentissage est souvent la **vitesse de simulation**, pas le réseau de neurones. Un moteur physique massivement parallèle (type Isaac Gym) accélère l'apprentissage bien plus qu'un réseau plus gros.

Partie 6 : Modèles du monde — quand le modèle remplace la physique

Une fonction de transition, apprise

Jusqu'ici, la physique était **calculée** : contacts, contraintes, intégration. Un **modèle du monde** prend le problème par l'autre bout — il **apprend** la fonction de transition à partir d'exemples.

$$s_{t+1} = f_{\theta}(s_t, a_t)$$

f_{θ} est un réseau de neurones, entraîné sur des triplets (s_t, a_t, s_{t+1}) enregistrés en faisant tourner un moteur physique, ou en observant le monde réel. À l'usage, il **remplace** le solveur : on lui donne un état et une action, il rend l'état suivant.

Question	Moteur physique	Modèle appris
D'où vient la règle ?	Des lois de Newton, écrites par un humain	Des données, ajustées par optimisation
Coût d'un pas	Broad phase, narrow phase, N itérations	Quelques multiplications matricielles
Garanties	Conservation de l'énergie, contraintes exactes	Aucune — seulement ce que les données montrent
Domaine de validité	Tout ce qui est modélisé	Ce qui ressemble à ses données
Où il excelle	Précision, cas rares, robustesse	Vitesse, parallélisme, effets mal modélisés

Ce n'est pas une opposition. Le moteur reste la **référence** — c'est lui qui fournit les données et qui sert à vérifier. Le modèle est une **approximation** qui achète de la vitesse et du parallélisme avec de la garantie.

Modèle du monde : remplacer le solveur par un réseau

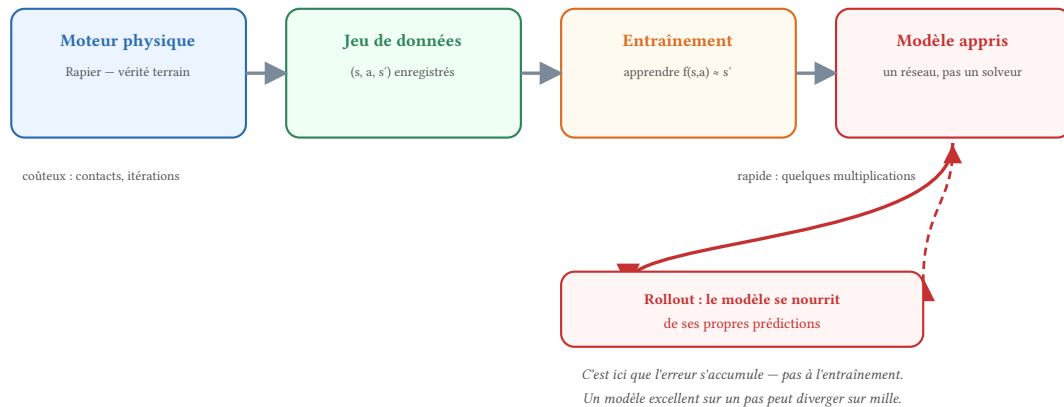


Figure 7: Le passage du solveur au modèle. Le moteur physique sert à **produire** les données, l'entraînement ajuste f_θ , puis le modèle est utilisé seul. La boucle rouge est le point sensible : en roulage, la sortie du modèle devient son entrée, donc chaque erreur est réinjectée dans la suivante.

⚠ Le problème central : l'erreur s'accumule

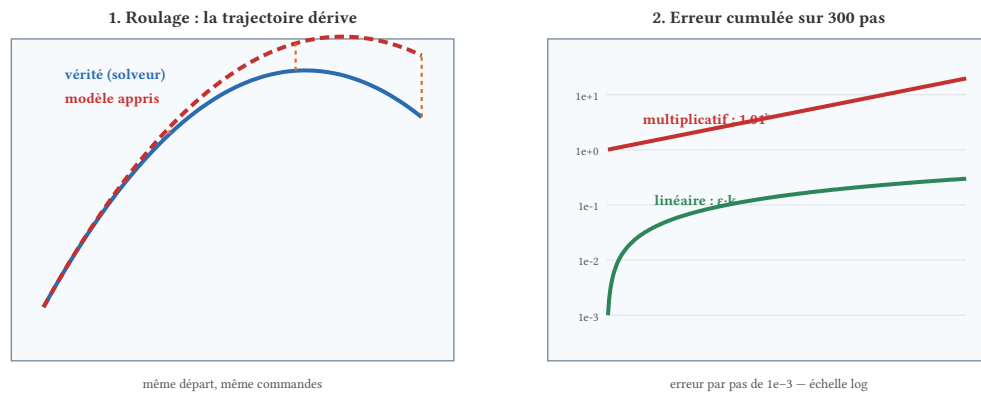
À l'entraînement, on minimise l'erreur sur **un** pas. À l'usage, on enchaîne des centaines de pas — et la sortie devient l'entrée.

Prenons une erreur de 10^{-3} par pas :

Régime	Erreur après 300 pas	Conséquence
Linéaire (εk)	$10^{-3} \times 300 = 0.3$	Dérive visible : le personnage glisse à côté
Multiplicatif (1.01^k)	$1.01^{300} \approx 20$	Divergence : le système explose

Un modèle **excellent** sur un pas peut donc être inutilisable sur mille. C'est la différence essentielle avec un solveur : un moteur physique ne **dérive** pas — il peut être imprécis, mais il reste stable, parce qu'il respecte les contraintes à chaque pas.

Le problème central : l'erreur s'accumule



Une erreur de $1e-3$ par pas devient 0.3 en 300 pas (linéaire),
ou $\times 20$ (multiplicatif). Aucune des deux n'est acceptable pour un long roulage.

Figure 8: À gauche, deux roulages partant du même état avec les mêmes commandes : le solveur (bleu) et le modèle (rouge). L'écart se creuse progressivement. À droite, la même erreur de 10^{-3} par pas en échelle logarithmique : selon qu'elle s'additionne ou se multiplie, on obtient une dérive ou une divergence.

Encoder la physique dans l'architecture

Un réseau dense n'a aucune raison de respecter une loi de conservation : rien ne l'empêche de **créer** de l'énergie. La parade la plus efficace n'est pas de pénaliser l'erreur davantage, mais de **changer la structure** du modèle pour que la loi soit vraie par construction.

Structure	Ce qu'elle impose
Graph network	Les objets sont des nœuds, les interactions des arêtes : la prédiction est locale et se généralise au nombre d'objets
Hamiltonien appris	Le réseau prédit une énergie, pas une accélération : la conservation découle de la dérivée
Lagrangien appris	Le réseau prédit un lagrangien, les équations du mouvement en découlent
Équation différentielle neuronale	Le réseau prédit une dérivée, l'intégration est faite par un solveur classique
Simulateur différentiable	Le solveur reste physique, mais on peut dériver à travers lui

C'est le même principe que les contraintes de la Session 8 : plutôt que d'espérer qu'une optimisation trouve la bonne solution, on **restreint l'espace des possibles** pour que la bonne solution soit la seule accessible.

Modèles latents : apprendre dans un rêve

Une variante très utilisée en RL n'essaie pas de prédire l'état **complet** — trop grand, trop bruité. Elle apprend d'abord un **état latent** compact (par exemple par un auto-encodeur), puis un modèle de transition **dans cet espace latent**.

1. Un encodeur transforme l'observation en un vecteur latent z_t .
2. Un modèle prédit z_{t+1} à partir de (z_t, a_t) .
3. Une politique est entraînée **à l'intérieur** du modèle — « dans le rêve » — sans toucher au vrai environnement.
4. Périodiquement, on revient au vrai environnement pour corriger la dérive du modèle.

L'idée forte : le modèle devient le **terrain d'entraînement** de la politique. On peut alors faire des milliers d'épisodes imaginaires pour le prix de quelques épisodes réels. C'est ce que fait la famille **Dreamer**, et c'est aujourd'hui l'une des approches les plus efficaces pour apprendre avec peu d'interactions réelles.

La limite reste la même : si le rêve dérive, la politique apprend à réussir dans un monde qui n'existe pas.

Approches hybrides — corriger le solveur

Entre « tout physique » et « tout appris », il y a une position pragmatique : garder le solveur et n'apprendre que ce qu'il rate.

Approche	Principe
Physique résiduelle	Le solveur fait le gros du travail ; un réseau apprend la correction Δs restante
Frottement appris	Le contact est résolu physiquement, mais le coefficient de frottement vient d'un modèle
Surrogate de simulation	Remplacer un solveur coûteux (tissu, fluide) par un réseau entraîné hors ligne
Simulateur différentiable	Garder la physique exacte, mais rendre le pas dérivable pour l'optimisation

La **physique résiduelle** est souvent la meilleure affaire : le réseau n'a qu'une petite correction à apprendre, donc moins de données et moins de dérive. Le solveur continue de garantir la stabilité — c'est lui qui empêche l'explosion.

💡 Quand un modèle du monde vaut la peine

- **La simulation est trop lente** : tissu, fluide, chevelure — un surrogate entraîné hors ligne peut suffire en jeu.
- **Le modèle physique est faux** : frottement réel, déformation de matériau, aérodynamique — un réseau peut capturer ce que les équations ne décrivent pas.
- **On veut entraîner une politique** : le modèle sert de terrain d'entraînement rapide et parallèle.
- **On veut dériver à travers la physique** : un simulateur différentiable permet d'optimiser des paramètres par gradient.

À l'inverse, un modèle appris est un mauvais choix quand la **garantie** compte plus que la vitesse : collision à ne jamais manquer, contrainte de sécurité, système à longue durée. Un solveur peut être lent ; il ne dérive pas.

Partie 7 : Particules sur GPU

Pourquoi le GPU peut aider

Pour une simulation de particules, chaque particule peut souvent mettre à jour sa position et sa vitesse **indépendamment** des autres. On peut donc exécuter le même petit programme sur des milliers d'éléments en parallèle.

Critère	CPU	GPU
Cœurs	Quelques-uns à quelques dizaines	Des milliers
Modèle	Exécution indépendante par cœur	Groupe de threads exécutant la même instruction
Point fort	Logique complexe, branchements, E/S	Calcul régulier sur beaucoup de données
Faiblesse	Débit limité	Branchements divergents, latence, synchronisation

Le GPU n'est pas « plus rapide » : il est **large**. Il gagne quand on lui donne beaucoup de travail identique et indépendant.

Le vrai coût : les transferts

La mémoire du CPU et celle du GPU sont séparées. Envoyer des données à chaque frame coûte cher. C'est exactement le problème rencontré en Session 9 avec `needsUpdate` : on modifiait un `Float32Array` côté CPU et il fallait le ré-uploader.

L'intérêt du compute shader est de supprimer ce transfert : l'état des particules reste sur la carte, et seul le **résultat** est dessiné. Le CPU n'envoie plus que quelques paramètres (pas de temps, gravité, nombre d'émissions).

Donnée	Fréquence	Taille typique
Paramètres globaux	Une fois par frame	Quelques dizaines d'octets
État des particules	Jamais renvoyé au CPU	Quelques Mo en VRAM

Un compute shader, en WGSL.

```
struct Particule {
    pos : vec3f,
    age : f32,
    vel : vec3f,
    vie : f32,
};

@group(0) @binding(0) var<storage, read_write> particules : array<Particule>;
```

```

@group(0) @binding(1) var<uniform> params : Params;

@compute @workgroup_size(256)
fn mise_a_jour(@builtin(global_invocation_id) gid : vec3u) {
    let i = gid.x;
    if (i >= arrayLength(&particules)) { return; } // borne du dernier groupe

    var p = particules[i];
    p.vel += params.gravite * params.dt;
    p.pos += p.vel * params.dt;
    p.age += params.dt;

    if (p.age >= p.vie) {
        p = reinitialiser(i); // nouvelle particule
    }
    particules[i] = p;
}

```

Trois choses à observer :

- Le **groupe de travail** traite 256 indices ; plusieurs groupes couvrent le tableau.
- La **vérification de borne** est indispensable : le dernier groupe contient presque toujours des threads « en trop ».
- Le shader **écrit** dans le buffer (read_write) : c'est ce qui permet à l'état de vivre sur la carte d'une frame à l'autre.

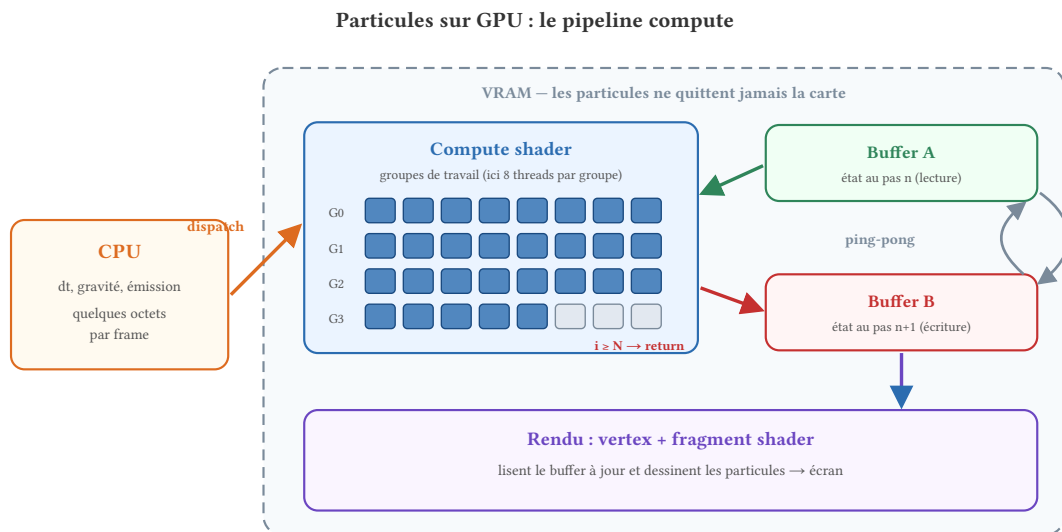


Figure 9: Le pipeline compute. Le CPU n'envoie que quelques paramètres ; les particules vivent en VRAM. Les groupes de travail traitent des blocs de 256 indices — et le **dernier** groupe contient presque toujours des threads hors bornes (en gris), d'où le test $i \geq N$. Le ping-pong évite qu'un thread lise ce qu'un autre est en train d'écrire.

Le même calcul, en GLSL (OpenGL 4.3+).

```

layout(local_size_x = 256) in;

struct Particule { vec4 posAge; vec4 velVie; }; // alignement 16 octets

layout(std430, binding = 0) buffer Particules { Particule particules[]; };
layout(std140, binding = 1) uniform Params { float dt; float gravite; };

```

```

void main() {
    uint i = gl_GlobalInvocationID.x;
    if (i >= particules.length()) return;

    Particule p = particules[i];
    p.velVie.xyz += vec3(0.0, -gravite, 0.0) * dt;
    p.posAge.xyz += p.velVie.xyz * dt;
    p.posAge.w += dt;

    if (p.posAge.w >= p.velVie.w) { p = reinitialiser(i); }
    particules[i] = p;
}

```

En GLSL, on regroupe souvent les données en vec4 pour respecter l'alignement mémoire — un détail d'implémentation, pas de physique. Le **concept** reste identique.

⚠ WebGL 2 n'a pas de compute shaders

C'est une limite d'API, pas de matériel. WebGL 2 expose les shaders de rendu (vertex, fragment) mais pas les shaders génériques de calcul. Pour un vrai compute shader dans un navigateur, il faut une API qui le prend en charge — WebGPU, ou du WebGL 2 détourné par des techniques de **transform feedback** et de rendu dans une texture.

La Session 9 utilise WebGL avec la physique sur le CPU : c'est le compromis normal pour du WebGL, et il est suffisant pour quelques milliers de particules. Le compute shader devient intéressant au-delà.

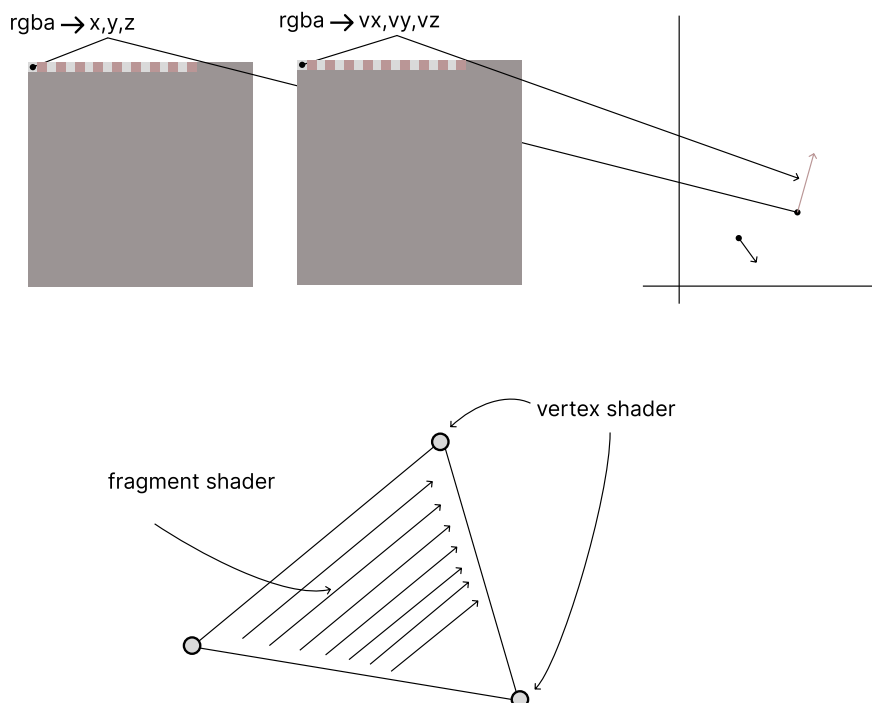


Figure 10: Une astuce historique pour simuler sans compute shader : **encoder l'état dans une texture**. La position et la vitesse de chaque particule deviennent des pixels ; un shader de rendu vers une texture applique la mise à jour, et deux textures alternées remplacent les buffers ping-pong. C'est la même idée que le pipeline précédent, détournée par le pipeline graphique.

Synchronisation et ping-pong

Un compute shader qui lit et écrit le **même** buffer peut créer des lectures et écritures concurrentes. Deux solutions courantes :

- **Barrière** : imposer une synchronisation entre l'écriture et la lecture suivante, quand l'API la fournit.
- **Ping-pong** : deux buffers A et B ; on lit A et on écrit B, puis on échange. Coût mémoire doublé, indépendance totale des lectures.

Le ping-pong est la solution la plus portable : elle ne dépend pas des garanties de synchronisation de l'API.

Quand les particules interagissent : grille spatiale

Tant que les particules sont indépendantes, le parallélisme est trivial. Dès qu'elles **interagissent** — collision, pression, voisinage — chaque particule doit connaître ses voisines.

Comparer toutes les paires coûte $O(N^2)$: pour 10 000 particules, cent millions de comparaisons par pas. La parade standard est la **grille spatiale** :

1. Découper l'espace en cellules de taille comparable au rayon d'interaction.
2. Ranger chaque particule dans une cellule (un tri par clé).
3. Ne comparer qu'avec les particules des cellules voisines.

C'est le même esprit que le **broad phase** de la Session 6 — sauf qu'ici il s'exécute sur le GPU. On obtient $O(N)$ en pratique, au prix d'un tri et d'une indirection mémoire.

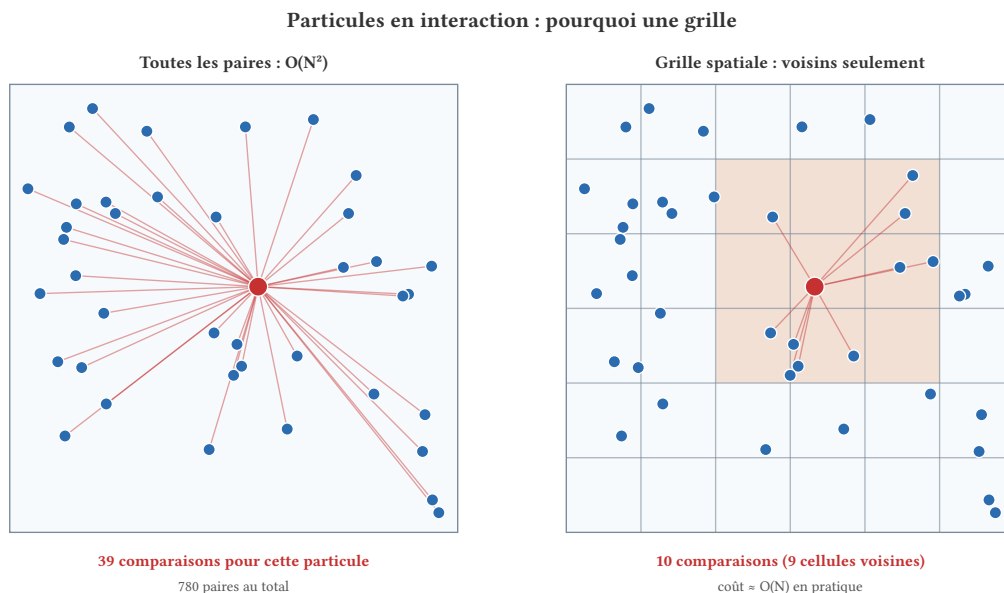


Figure 11: Même particule (en rouge), deux stratégies. À gauche, elle se compare à **toutes** les autres : 39 comparaisons pour cette seule particule, 780 paires pour l'ensemble. À droite, elle ne regarde que les 9 cellules voisines : 10 comparaisons. Le gain n'est pas une accélération matérielle, c'est une réduction du **nombre de comparaisons** — le GPU ne fait que rendre la grille plus rapide encore.

⚠ Le GPU n'est pas une accélération magique

- **Mesurer avant d'optimiser.** Pour quelques centaines de particules, le coût du dispatch et de la synchronisation peut dépasser le calcul lui-même.
- **Les branchements coûtent.** Dans un groupe de threads, si les chemins divergent, le matériel exécute les deux. Un shader plein de `if` disparates est lent.
- **La mémoire est le goulot.** Un accès dispersé (indirection, grille) peut annuler le gain du parallélisme.
- **Le solveur rigide n'est pas un cas de particules.** Les contacts entre corps rigides forment un système couplé ; il ne se parallélise pas aussi simplement qu'un nuage de points indépendants.

Physique GPU dans les moteurs

Outil	Approche
PhysX (GPU)	Simulation rigide massivement parallèle, pour les scènes à grand nombre de corps
Flex / Unified Particles	Particules unifiées : fluides, tissus et solides déformables dans le même solveur
Isaac Gym / Isaac Lab	Milliers d'environnements simulés sur GPU, conçus pour l'apprentissage
Houdini / Blender	Simulation hors ligne de haute qualité, bakée ensuite en animation

Deux usages très différents : le **temps réel** (jeu) et l'**entraînement** (apprentissage). Le second accepte des compromis de précision que le premier refuse.

Partie 8 : Pièges et bonnes pratiques

Les cinq erreurs les plus fréquentes

1. **Confondre observation et état.** L'agent ne voit pas le monde : il voit ce qu'on lui a encodé. Si une information manque, aucune quantité d'entraînement ne la fera apparaître.
2. **Récompense mal conçue.** L'agent optimise exactement ce qui est écrit. Un terme oublié devient une stratégie.
3. **Conditions d'évaluation non comparables.** Si chaque génome a un départ différent, on sélectionne la chance, pas la compétence.
4. **Optimiser sans mesurer.** Lancer un compute shader sur 500 particules est souvent plus lent que la boucle CPU équivalente.
5. **Croire que la simulation est la réalité.** Une politique qui marche en simulation échoue souvent dans le monde réel : masses, frottements et délais différents.

Méthode de travail recommandée

1. **Établir une référence simple.** Une règle écrite à la main, ou une animation. Sans point de comparaison, impossible de dire si l'IA apporte quelque chose.
2. **Instrumenter.** Afficher les observations, les commandes et la récompense en direct. La plupart des problèmes se voient immédiatement.

3. **Simplifier avant d'agrandir.** Un réseau plus gros ne corrige pas une observation manquante ni une récompense ambiguë.
4. **Tester hors des conditions d'entraînement.** C'est la seule mesure honnête de la robustesse.

Pour aller plus loin — références

Comment utiliser cette liste

Ces références sont données par **auteur, titre et année** — elles se retrouvent par leur titre dans un moteur de recherche ou sur un portail d'articles scientifiques. Elles sont classées par thème, du plus accessible au plus spécialisé. Pour un cours d'introduction, les trois premières de chaque section suffisent.

Reinforcement Learning — fondements

- Sutton & Barto, **Reinforcement Learning: An Introduction**, 2e édition, MIT Press, 2018. — La référence pédagogique. Les chapitres 1 à 4 couvrent tout ce qui précède.
- Bellman, **Dynamic Programming**, 1957. — L'origine de l'équation de valeur.
- Watkins & Dayan, **Q-learning**, Machine Learning, 1992. — L'algorithme de l'exemple du couloir.
- Mnih et al., **Human-level control through deep reinforcement learning**, Nature, 2015. — DQN : le réseau remplace la table.
- Schulman et al., **Trust Region Policy Optimization**, 2015.
- Schulman et al., **Proximal Policy Optimization Algorithms**, 2017. — L'algorithme le plus utilisé en locomotion.
- Lillicrap et al., **Continuous control with deep reinforcement learning**, 2015. — DDPG, pour les actions continues.
- Haarnoja et al., **Soft Actor-Critic**, 2018.
- Silver et al., **Mastering the game of Go with deep neural networks and tree search**, Nature, 2016.

Personnages et locomotion physique

- Yin, Loken, van de Panne, **Simbicon: Simple Biped Locomotion Control**, SIGGRAPH 2007. — Le contrôleur procédural, point de départ historique.
- Coros et al., **Locomotion Skills for Simulated Quadrupeds**, SIGGRAPH 2011.
- Peng, Abbeel, Levine, van de Panne, **DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills**, SIGGRAPH 2018. — Imitation de captures de mouvement.
- Peng et al., **Learning Agile Robotic Locomotion Skills by Imitating Animals**, RSS 2020.
- Lee et al., **Learning Quadrupedal Locomotion over Challenging Terrain**, Science Robotics, 2020.
- Rudin, Hoeller, Reist, Hutter, **Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning**, CoRL 2021. — Le lien direct avec le parallélisme massif.
- Liu & Hodgins, **Learning to Schedule Control Fragments for Physics-Based Characters Using Deep Q-Learning**, ACM TOG, 2017.

Évolution et neuroevolution

- Holland, **Adaptation in Natural and Artificial Systems**, 1975. — L'origine des algorithmes génétiques.
- Goldberg, **Genetic Algorithms in Search, Optimization, and Machine Learning**, 1989. — Référence classique.
- Stanley & Miikkulainen, **Evolving Neural Networks through Augmenting Topologies**, Evolutionary Computation, 2002. — NEAT.
- Floreano, Husbands, Nolfi, **Evolutionary Robotics**, dans le Springer Handbook of Robotics, 2008. — Panorama.
- Such et al., **Deep Neuroevolution: Genetic Algorithms Are a Competitive Alternative for Training Deep Neural Networks for Reinforcement Learning**, 2017.
- Salimans et al., **Evolution Strategies as a Scalable Alternative to Reinforcement Learning**, 2017.

Simulation physique et apprentissage

- Todorov, Erez, Tassa, **MuJoCo: A physics engine for model-based control**, IROS 2012. — Le moteur historique du RL en robotique.
- Coumans & Bai, **PyBullet**, 2016–2021. — Moteur libre, très utilisé pour l'apprentissage.
- Makoviychuk et al., **Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning**, 2021. — La simulation sur GPU pour l'entraînement.
- Bender, Müller, Otaduy, Teschner, Macklin, **A Survey on Position Based Dynamics**, Eurographics STAR, 2017. — Le lien avec les Sessions 8 et 18.
- Müller et al., **Position Based Dynamics**, 2007. — L'article fondateur du PBD.

Modèles du monde et simulation apprise

- Ha & Schmidhuber, **World Models**, 2018. — L'article qui a popularisé l'entraînement d'une politique **à l'intérieur** d'un modèle appris.
- Hafner et al., **Dream to Control: Learning Behaviors by Latent Imagination**, 2019. — Dreamer : modèle latent, politique entraînée dans le rêve.
- Hafner et al., **Mastering Diverse Domains through World Models**, 2023. — DreamerV3, la version généraliste.
- Battaglia et al., **Interaction Networks for Learning about Objects, Relations and Physics**, NeurIPS 2016. — Prédire la physique avec des graphes d'objets.
- Sanchez-Gonzalez et al., **Learning to Simulate Complex Physics with Graph Networks**, ICML 2020. — Les **graph network simulators**, la référence actuelle.
- Greydanus, Dzamba, Yosinski, **Hamiltonian Neural Networks**, NeurIPS 2019. — Conserver l'énergie **par construction**.
- Cranmer et al., **Lagrangian Neural Networks**, 2020. — La même idée via le lagrangien.
- Chen et al., **Neural Ordinary Differential Equations**, NeurIPS 2018. — Le réseau prédit une dérivée, un solveur l'intègre.
- de Avila Belbute-Peres et al., **End-to-End Differentiable Physics for Learning and Control**, NeurIPS 2018.
- Hu et al., **DiffTaichi: Differentiable Programming for Physical Simulation**, ICLR 2020.
- Freeman et al., **Brax: A Differentiable Physics Engine for Large Scale Rigid Body Simulation**, 2021.

- Silver et al., **Residual Policy Learning**, 2018. — L'idée de n'apprendre que la **correction** d'un contrôleur existant.
- Bertiche, Madadi, Escalera, **Neural Cloth Simulation**, SIGGRAPH Asia 2020. — Remplacer un solveur de tissu par un réseau.

Particules, fluides et GPU

- Stam, **Stable Fluids**, SIGGRAPH 1999. — Les fluides incompressibles en temps réel.
- Macklin & Müller, **Position Based Fluids**, SIGGRAPH 2013. — Les fluides en PBD, sur GPU.
- Macklin et al., **Unified Particle Physics for Real-Time Applications**, SIGGRAPH 2014. — Fluides, tissus et déformables dans un seul solveur.
- Nguyen (éd.), **GPU Gems 3**, NVIDIA, 2007. — Chapitres sur les systèmes de particules et le calcul parallèle.
- Harris et al., **Parallel Prefix Sum (Scan) with CUDA**, dans GPU Gems 3, 2007. — La primitive qui structure beaucoup d'algorithmes GPU.
- NVIDIA, **CUDA Programming Guide**. — Le modèle de programmation, utile pour comprendre les groupes de threads.

API, spécifications et outils

- W3C, **WebGPU Specification** et **WGSL Specification**. — Les deux documents qui définissent le compute shader dans le navigateur.
- Khronos, **OpenGL 4.3** — introduction des compute shaders et des **shader storage buffer objects**.
- Documentation de **Rapier** (rapier.rs) — joints, contrôleurs de véhicule, requêtes de scène.
- Documentation de **Three.js**, dossier exemples/ — démos WebGPU de particules, calcul GPU.
- Documentation de **Blender** (docs.blender.org) — cloth, soft body, simulation et bake.
- Documentation de **Typst** (typst.app/docs) — pour la mise en forme des rapports de projet.



Deux lectures si vous ne lisez qu'une chose par thème

- **Pour le RL** : le chapitre 1 de Sutton & Barto. Il pose le vocabulaire utilisé dans toute la suite du domaine.
- **Pour la physique de personnage** : DeepMimic. C'est l'article qui montre le mieux comment une récompense bien conçue remplace un contrôleur écrit à la main.

Où suivre la recherche

Domaine	Conférences et revues
Synthèse d'images	SIGGRAPH, SIGGRAPH Asia, Eurographics, ACM TOG (revue)
Animation de personnages	SCA (Symposium on Computer Animation), MIG (Motion, Interaction and Games)
Apprentissage automatique	NeurIPS, ICML, ICLR
Robotique et apprentissage	ICRA, IROS, RSS, CoRL

Domaine	Conférences et revues
Industrie du jeu	GDC (Game Developers Conference)

Ces lieux sont stables depuis des années : suivre leurs actes est le moyen le plus fiable de voir où va le domaine. Pour un projet étudiant, les actes de **SIGGRAPH**, de **SCA** et de **CoRL** sont les plus proches du contenu de ce cours — graphique, animation et apprentissage pour la robotique.

Annexe — glossaire

Vocabulaire du Reinforcement Learning

Terme	Définition
État s_t	Tout ce qui détermine la suite du monde.
Observation o_t	Ce que l'agent perçoit — souvent un sous-ensemble de l'état.
Action a_t	Commande envoyée au monde : force, couple, braquage, cible de joint.
Récompense r_t	Note scalaire immédiate attribuée à une transition.
Retour G_t	Somme actualisée des récompenses futures depuis l'instant t .
Facteur γ	Poids du futur : 0 = myope, proche de 1 = prévoyant.
Politique π	Règle qui associe une action (ou une distribution) à une observation.
Valeur V^π, Q^π	Espérance de retour depuis un état, ou depuis un état et une action.
Épisode	Une exécution complète, du départ à la fin (chute, arrivée, temps écoulé).
ε -glouton	Prendre la meilleure action, sauf avec probabilité ε où l'on explore.

Vocabulaire de l'évolution

Terme	Définition
Génome	Vecteur des paramètres d'une politique — ici les poids et biais du réseau.
Population	Ensemble de génomes évalués à la même génération.
Fitness	Score d'un génome sur un épisode ; sert à classer et sélectionner.
Sélection	Choix des génomes qui se reproduiront (troncature, tournoi, roulette).
Élitisme	Conserver les meilleurs sans les modifier d'une génération à l'autre.

Terme	Définition
Mutation	Perturber aléatoirement des gènes : $w_{i'} = w_i + \varepsilon$.
Croisement	Combiner les gènes de deux parents.
Génération	Un cycle complet : évaluer, sélectionner, reproduire.
Convergence	Perte de diversité quand la population devient identique — souvent trop tôt.

Vocabulaire du GPU et de la simulation

Terme	Définition
Compute shader	Programme exécuté sur le GPU hors du pipeline de rendu, pour du calcul général.
Groupe de travail	Bloc de threads lancés ensemble (par ex. 256) partageant des données locales.
Dispatch	Lancement du compute shader sur un nombre donné de groupes de travail.
SSBO / storage buffer	Buffer de grande taille accessible en lecture et écriture par un shader.
Ping-pong	Deux buffers alternés : on lit A et on écrit B, puis on échange.
Barrière	Synchronisation imposant qu'une étape soit terminée avant la suivante.
Modèle du monde	Réseau qui prédit l'état suivant à partir de l'état et de l'action, au lieu de le calculer.
Rollout	Enchaînement de prédictions où la sortie du modèle devient son entrée — là où l'erreur s'accumule.
État latent	Représentation compacte apprise, dans laquelle on fait tourner le modèle pour économiser du calcul.
Biais inductif	Structure imposée au modèle (graphe, hamiltonien...) pour que les lois physiques soient vraies par construction.
Simulateur différentiable	Solveur physique à travers lequel on peut dériver, pour optimiser par gradient.
Physique résiduelle	Le solveur calcule le principal, un réseau n'apprend que la correction restante.
Grille spatiale	Découpage de l'espace en cellules pour ne comparer que des voisins proches.
Bake	Calcul d'une simulation hors ligne, enregistré comme animation réutilisable.
Sim-to-real	Transfert d'une politique entraînée en simulation vers le monde réel.

Terme	Définition
Randomisation de domaine	Variation des paramètres physiques à l'entraînement pour gagner en robustesse.

💡 Trois questions pour se repérer

1. « **Qui décide ?** » → la politique. « **Qui calcule les conséquences ?** » → la physique.
2. « **Comment la politique s'améliore-t-elle ?** » → par gradient (RL) ou par sélection (évolution).
3. « **Qui calcule la physique ?** » → un solveur (garanti, mais coûteux) ou un modèle appris (rapide, mais qui dérive).
4. « **Où le calcul a-t-il lieu ?** » → CPU pour la logique et le solveur rigide, GPU pour un grand nombre d'éléments indépendants.

À retenir

💡 Les cinq idées de la session

- La **physique** simule les conséquences ; la **politique** choisit les commandes. Le réseau ne contourne jamais les forces ni les contacts.
- En **RL**, l'agent apprend à partir d'observations, d'actions et de récompenses, en optimisant un retour actualisé par γ .
- En **neuroevolution**, on fait évoluer les paramètres d'une **population** de politiques : sélection, mutation, élitisme — sans gradient.
- Les **observations** et la **récompense** définissent le problème. Un mauvais signal produit une mauvaise stratégie, même si l'algorithme fonctionne parfaitement.
- Le **GPU** est utile quand beaucoup de calculs sont indépendants et que les données restent sur la carte. Ce n'est ni automatique, ni universel — et le goulot d'étranglement est souvent la mémoire, pas le calcul.

💡 Vers la Session 20

La Session 20 est une séance de révision suivie d'un quiz final. Les notions de cette session qui s'y prêtent le mieux sont des **distinctions** :

- **état** contre **observation** — ce que le monde est, contre ce que l'agent en perçoit ;
- **RL** contre **neuroevolution** — apprendre par gradient, ou par sélection ;
- **calcul CPU** contre **calcul GPU** — la logique et le solveur d'un côté, le parallélisme massif de l'autre.

Ce sont des questions de vocabulaire autant que de technique, et c'est précisément ce qu'un quiz peut vérifier. Revoyez aussi les deux exemples chiffrés — le couloir à quatre cases et la comparaison de fitness : ils contiennent l'essentiel du raisonnement.